

CS254 Advanced Database Systems

Lab 2 (20%): Replacing the MiniOB MemTable's Ordered Structure with a B+-tree

Overview

In MiniOB/ObLsm, the MemTable is an in-memory ordered component used to buffer incoming write operations before they are flushed to SSTables.

In the current implementation, the MemTable uses a skiplist as its data structure.

In this lab, you need to study the existing MemTable design and replace its internal ordered structure with an **in-memory B+-tree**.

Learning Objectives

By completing this lab, you should be able to:

- understand the role of the MemTable in an LSM-tree storage engine;
- compare skiplist and B+-tree as ordered data structures;
- implement and validate a non-trivial storage engine modification.

Reading Materials

Before starting the lab, read the following materials:

1. **MiniOB LSM-Tree design document** <https://oceanbase.github.io/miniob/design/miniob-lsm-tree/>
2. **MiniOB B+-tree design document (for reference only)**
<https://oceanbase.github.io/miniob/design/miniob-bplus-tree/> Note: this document describes a page/file-oriented B+-tree index, not an in-memory MemTable implementation.
3. **Relevant source directory**

```
src/oblsm/
```

You are expected to identify the MemTable-related classes and the code paths for lookup, iteration, dump, freeze, and flush.

Fixed Starter Version

All submissions must be based on the following MiniOB starter commit:

```
9f856a542decb6dc678650406af7d6e351940dab
```

Your patch file must be generated **against this exact commit**.

Submissions generated against any other version may fail to apply and may receive no credit.

Tasks

Task 1. Understand the Original MemTable

Read the design document and source code, and understand:

- the role of the MemTable in the Oblsm engine;
- how records are stored in the MemTable;
- how ordered iteration is used by `MemTableIterator` and during flushing.

Task 2. Replace the Skiplist-based MemTable with a B+-tree

The current MemTable uses a skiplist as its internal ordered structure. Your task is to replace that ordered structure with an **in-memory B+-tree**.

2.1 B+-tree configuration

Your B+-tree must expose the following two tunable parameters:

- `internal_max_children`: the maximum number of **child pointers** in an internal node. Therefore, an internal node can contain at most `internal_max_children - 1` separator keys.
- `leaf_max_entries`: the maximum number of entries stored in a leaf node.

You may implement these parameters as constants, constructor arguments, or configuration variables. State clearly in your report how they are configured in your implementation.

2.2 Structural statistics and logging

You should also add a structure to count:

- leaf split counts;
- internal-node split counts;
- the resulting tree height.

Whenever an operation causes a **split**, you must print one INFO-level log entry. For the default log related configuration, refer to `etc/observer.ini`.

Use the following format exactly:

```
Split. <leaf split counts> <internal-node split counts> <the resulting tree height>
```

If the initial root is a leaf node and it splits into two leaf nodes with a newly created root, count this as a **leaf split**, not an internal-node split.

Note: Think about why this lab does not require you to report node merge counts.

2.3 Functional requirements

Your modified MemTable must satisfy the following requirements:

- the code must compile successfully, and a submission that cannot be compiled or executed will not be graded;
- the MemTable must preserve the original key ordering and versioning semantics;
- the original MemTable behavior must be preserved;
- ordered iteration must continue to work correctly;
- the modified MemTable must continue to support downstream dump / freeze / flush logic correctly.

2.4 Implementation constraints

- You may add new classes, helper functions, member variables, and any other supporting code as needed.
- You are free to redesign the internal implementation of the MemTable, as long as the required functionality is correctly implemented and the modified code works properly within the existing MiniOB framework.
- You do **not** need to preserve the exact internal structure of the original skiplist-based implementation.
- You must implement an **in-memory B+-tree** for the MemTable.
- The existing MiniOB B+-tree implementation may be used as a design reference, but it is **not** a drop-in solution for this lab. Please figure out the reason by yourself.
- Your B+-tree must preserve the **same comparator, ordering, and versioning semantics** currently used by the MemTable. In particular, do not replace the existing MemTable ordering with a simpler user-key-only ordering unless that is already the current behavior.

Task 3. Verify the Correctness of Your Implementation

You must test your implementation and include the results in your report.

The goal of testing is not only to show that your B+-tree works as a data structure, but also to verify that replacing the internal ordered structure does **not** break the original functionality of the MemTable or its interactions with the rest of ObLsm.

In particular, your tests should verify that the modified MemTable still preserves the original behavior described in the ObLsm design documents, including:

- correct handling of **select**, **insert**, **update**, and **delete** records;
- correct maintenance of record ordering in the MemTable;
- correct behavior for repeated writes to the same key;
- correct behavior of record encoding and lookup semantics;
- correct ordered traversal through **MemTableIterator**;
- correct interaction with downstream dump / freeze / flush logic.

At a minimum, your testing should cover the following cases:

1. Basic insertion and lookup

- Insert multiple key-value pairs into the MemTable.
- Verify that the inserted records can be found correctly.

2. Multiple writes to the same key

- Insert or update the same key multiple times with different values or timestamps.
- Verify that the MemTable still behaves correctly for repeated versions of the same user key.
- **Hint:** Refer to the MemTable section of the MiniOB LSM-Tree design document and the corresponding source files. Your B+-tree must preserve the **same ordering and versioning semantics** as the original MemTable.

3. Delete-related behavior

- Verify that delete-related records are handled correctly according to the existing MemTable semantics.
- In particular, if the current MemTable represents deletion using tombstone records, your replacement must preserve that behavior.

4. Ordered iteration

- Verify that `MemTableIterator` can still traverse records in the expected order after the replacement.

5. Dump / freeze / flush-related behavior

- Verify that replacing the internal ordered structure does not break logic that depends on the MemTable's ordered output.

You should also check whether your log information is correct in the log file (`observer.log` by default).

MiniOB already provides some test infrastructure in the `test` directory, including `case`, `integration_test`, and `sysbench`. You may use them in your validation.

For example, basic tests are located in:

```
test/case
```

You can run:

```
python3 miniob_test.py --test-cases=basic
```

to execute the basic test cases.

Task 4. Performance Evaluation

The TA will evaluate your implementation from two aspects.

4.1 B+-tree correctness check

The TA will prepare additional test cases and check whether your implementation behaves correctly and whether your INFO-level structural logs are correct.

To avoid interference from Lab 1, TA test cases for this lab will include only:

- `insert`
- `select`
- `delete`

commands.

4.2 Performance check

You must run sysbench and report the results.

Before you get started, first install sysbench. On Ubuntu, a simple way is:

```
sudo apt install sysbench
```

In your report, test the following **three parameter settings**:

- `internal_max_children = 10, leaf_max_entries = 10`
- `internal_max_children = 50, leaf_max_entries = 50`
- `internal_max_children = 100, leaf_max_entries = 100`

For each setting, run the following default MiniOB sysbench scripts:

- `miniob_delete.lua`
- `miniob_insert.lua`
- `miniob_select.lua`

The sysbench test scripts are located in:

```
test/sysbench
```

To run sysbench tests, start MiniOB in MySQL protocol mode:

```
./bin/observer -f <config file> -P mysql -p <port>
```

Then, in another terminal, run sysbench:

```
sysbench <test_case> --db-driver=mysql --mysql-host=127.0.0.1 --mysql-port=<port>  
--mysql-db=test --tables=1 --table_size=1 <operation>
```

A simple example:

```
./bin/observer -f ../etc/observer.ini -P mysql -p 6789
```

In another terminal:

```
cd test/sysbench
sysbench miniob_insert.lua --db-driver=mysql --mysql-host=127.0.0.1 --mysql-
port=6789 --mysql-db=test --tables=1 --table_size=1 prepare
```

For the actual performance experiments required in this lab, use:

- `table_size = 1000`
- `tables = 5`
- `time = 60`

You should report both:

- sysbench performance results;
- Last node split information from your INFO logs.

Important note: Normally, `prepare` creates tables, `run` executes the workload, and `cleanup` removes tables. However, MiniOB currently does not implement `DROP TABLE`, so `cleanup` may not work correctly. You may either:

- implement `DROP TABLE` yourself; or
- manually remove the database files after MiniOB exits.

By default, database files are stored in:

```
build/miniob/db/sys
```

Make sure each experiment starts from a **clean database**.

You are strongly encouraged to run your own validation before submission. For more information, refer to [How to run miniob](#).

Task 5. Write a Report

Your report should include the following sections.

5.1 Design and Implementation

Describe your design decisions and implementation details:

- design overview;
- key data structures;
- important classes and functions you implemented or modified;
- how your B+-tree integrates with the MemTable.

5.2 Correctness Verification

Document your testing and verification approach:

- test cases used;
- expected results vs. actual results;
- evidence that MemTable semantics are preserved;
- sysbench performance results.

5.3 Discussion and Conclusions

Discuss the following points:

- challenges encountered and how you resolved them, if any;
- a comparison between skiplist-based and B+-tree-based MemTable designs;
- the effect of different node sizes on performance.

How to Generate Your Diff File

You must generate your patch file using `git diff` against the fixed starter commit:

```
git diff 9f856a542decb6dc678650406af7d6e351940dab > <student_id>.diff
```

Important Notes

1. Your work must be based on commit:

```
9f856a542decb6dc678650406af7d6e351940dab
```

2. Generate the patch.

If you have staged changes (i.e., you ran `git add`) and want to include only them, use:

```
git diff --cached 9f856a542decb6dc678650406af7d6e351940dab > <student_id>.diff
```

Otherwise, to include all **unstaged changes** in your working directory, use:

```
git diff 9f856a542decb6dc678650406af7d6e351940dab > <student_id>.diff
```

3. Do **not** submit the entire project directory or a zip file of the repository. Only submit:

- `<student_id>.diff`
- `<student_id>-report.pdf`

4. You are strongly encouraged to verify that your diff can be applied cleanly to a fresh checkout of the starter commit before submission.

Recommended Self-Check

Before submission, test your patch on a clean copy of the repository:

```
git checkout 9f856a542decb6dc678650406af7d6e351940dab
git apply --check <student_id>.diff
git apply <student_id>.diff
```

Then build and test your code.

If `git apply --check` fails, your submission is likely invalid.

How the TA Will Apply and Grade Your Patch

The TA will grade your submission using the following procedure:

1. Check out the fixed starter commit:

```
git checkout 9f856a542decb6dc678650406af7d6e351940dab
```

2. Verify that the patch can be applied:

```
git apply --check <student_id>.diff
```

3. Apply the patch:

```
git apply <student_id>.diff
```

4. Build the project and run tests.

Submissions may receive reduced or no credit if:

- the patch cannot be applied cleanly to the fixed starter commit;
- the project does not compile after the patch is applied;
- the submitted implementation does not satisfy the lab requirements.

For correctness evaluation, the TA will generate test cases containing only `insert`, `select`, and `delete` commands, and will check whether your INFO-level output is correct.

Submission

Submit the following files to Gradescope:

- `<student_id>.diff`
- `<student_id>-report.pdf`

where `<student_id>` is your student ID.

Only these two files should be submitted.

Write your report in **ENGLISH**.

Grading

This lab is graded primarily on:

- the correctness and completeness of your MemTable replacement;
 - code quality and consistency with the existing framework;
 - successful patch application and build;
 - the quality and clarity of the report.
-

Deadline

2026-04-26 23:59 (UTC+8)

Only the last active submission before the deadline will be graded.

Academic Integrity

This is an individual lab. All work must be your own. Collaboration is not allowed.

Any academic misconduct, including undeclared or inappropriate use of generative AI, may result in failure of the course.